

BankManagement

Database and Software Architecture Report

Design, Implementation, Security, and System Integration

Technology stack

Microsoft SQL Server and T-SQL; Node.js and Express; the `mssql` SQL Server driver; vanilla HTML, CSS, and JavaScript.

Implementation approach

The database contains the main business rules through tables, keys, constraints, stored procedures, functions, triggers, views, database roles, and Agent jobs. The backend calls these SQL objects directly; it does not use an object-relational mapper.

Student: _____

Course: Database Systems

Instructor: _____

Date: _____

Executive Summary

This report describes the design and implementation of the BankManagement project. The main emphasis is the SQL Server database: the relational schema, integrity rules, stored procedures, views, triggers, functions, security roles, reports, audit records, and scheduled jobs. The report also explains how the Node.js/Express API communicates with SQL Server and how the browser interface uses the API.

The current implementation contains seventeen relational tables, fifty-two stored procedures, fifteen reporting and helper views, six integrity triggers, two scalar functions, five database-security personas, and three SQL Server Agent jobs. Application identities are separate from SQL Server logins. The application uses four effective roles: Customer, Employee, Admin, and HighAdmin. A stored role is not accepted by itself; the session-validation procedure also checks the current customer status, employee status, job title, administrative capability, and active branch assignment.

System path in one line

Browser interface → Express route → session and role middleware → typed `mssql` parameters → stored procedure or protected view → SQL Server rules and audit → JSON response → role-aware interface.

Contents

Executive Summary	i
1 Project Scope and Requirements	1
1.1 Problem statement	1
1.2 Assignment requirements and implementation mapping	1
2 System Architecture	3
2.1 Layered architecture	3
2.2 Responsibility boundaries	3
2.3 Ports and CORS	3
3 Database Conceptual and Relational Design	4
3.1 Core banking ER model	4
3.2 Identity, role, and workforce ER model	6
3.3 Normalization to third normal form	8
4 Complete Data Dictionary	9
4.1 Customer	9
4.2 Branch	9
4.3 AccountType	10
4.4 Account	10
4.5 Employee	10
4.6 EMPB	11
4.7 TransactionType	11
4.8 Transactions	11
4.9 Loan	12
4.10 Installment	12
4.11 BranchLedger	13
4.12 Roles	13
4.13 Users	13
4.14 UserRoles	13
4.15 Sessions	14
4.16 AuditLog	14
4.17 EmployeeTransferRequest	14
5 Relationships, Keys, Constraints, and Indexes	16
5.1 Cardinality summary	16
5.2 Important indexes and why they exist	16
5.3 Account number generation	17

6	Database Business Rules and Integrity Automation	18
6.1	Account lifecycle	18
6.2	Available balance and pending holds	18
6.3	Transaction amount-dependent delay	18
6.4	Triggers	19
6.4.1	trg_EMPB_PreventDateOverlap	19
6.4.2	TR_Account_PreventCloseWithPending	19
6.4.3	TR_Account_SyncBranchLedger	19
6.4.4	TR_Customer_PreventDeactivateWithPending	19
6.4.5	TR_Transactions_SyncInstallmentStatus	19
6.4.6	TR_Transactions_EnforceTypePattern	19
6.5	Effective-role function	19
6.6	Transactional consistency and concurrency	19
7	Stored Procedures	20
7.1	Procedure design pattern	20
7.2	Authentication and session	20
7.3	Customer	20
7.4	Accounts	21
7.5	Transactions	22
7.6	Loans and installments	22
7.7	Branches, employees, and transfers	22
7.8	HighAdmin manager governance	23
7.9	Critical transaction flow	24
8	Views, Reports, and Scheduled Jobs	25
8.1	Reporting views	25
8.2	API report catalogue	25
8.3	Database pools and least privilege	26
8.4	SQL Server Agent jobs	26
9	Authentication, Authorization, Sessions, and Audit	28
9.1	Login and session sequence	28
9.2	How req.user is created	28
9.3	Assigned roles versus effective roles	29
9.4	Branch and ownership authorization	29
9.5	Auditing	29
10	SQL Injection Control and Direct SQL Data Access	30
10.1	Evidence that the project does not use an ORM	30
10.2	Difference between a driver and an ORM	31
10.3	SQL injection controls	31
11	Backend Architecture	32
11.1	Technology and startup	32
11.2	Configuration and database connection	32
11.3	Request handling pattern	32
11.4	API response shape	32
11.5	Complete API inventory	33
11.5.1	Setup, authentication, and customers	33

11.5.2	Accounts and transactions	33
11.5.3	Loans, employees, and transfers	34
11.5.4	Branches, HighAdmin governance, and reports	34
12	Frontend Architecture	36
12.1	Technology and file organization	36
12.2	Authentication page	36
12.3	Frontend session behavior	36
12.4	Role-aware workspaces	36
12.5	Dynamic dropdowns and branch overview	37
12.6	Date handling	37
13	End-to-End Use Cases	38
13.1	Customer opens an account	38
13.2	Manager views and acts on branch accounts	38
13.3	Employee hiring	38
13.4	Loan installment payment	38
14	Installation, Operation, and Demonstration	39
14.1	Database installation order	39
14.2	Run commands	39
14.3	Recommended live defense demonstration	39
15	Testing and Verification	40
15.1	Recommended additional tests	40
16	Limitations and Production Hardening	41
17	Questions Likely to Be Asked	42
A	Project File Map	44
A.1	Database	44
A.2	Backend	44
A.3	Frontend	44
B	Concise Relational Schema	45
C	Selected Code Excerpts	46
C.1	Server-defined report whitelist	46
C.2	Trusted identity overrides client identity	46
C.3	Fixed session expiration	46
	Presentation Summary	47

List of Tables

1.1 Requirement-to-implementation mapping 1

List of Figures

2.1	End-to-end architecture	3
3.1	Core banking relationships	5
3.2	Application identity, roles, workforce, and transfer workflow	7
6.1	Account status transitions	18
7.1	Financial transaction lifecycle	24
9.1	Authentication and session validation	28

Chapter 1

Project Scope and Requirements

1.1 Problem statement

The system models a multi-branch bank in which customers may own accounts at different branches, staff work through temporal branch assignments, financial operations are queued and finalized safely, loans are divided into installments, and access is controlled by both application roles and database-level permissions.

1.2 Assignment requirements and implementation mapping

Table 1.1: Requirement-to-implementation mapping

Requirement	Implementation
Relational database	SQL Server schema with normalized entity, lookup, bridge, workflow, and audit tables.
ER diagram and relational schema	Explicit PK/FK relationships, cardinalities, and temporal bridges documented in Chapters 3–5.
Customer management	Create, search, update, soft deactivate; staff results are branch scoped through customer accounts.
Account management	Self-service opening, visibility scopes, freeze/unfreeze, type change, close, dormant sweep, monthly interest.
Transactions	Owner-only deposit, withdrawal, transfer, finalization, cancellation/reversal; pending queue and batch processing.
Loans/installments	Branch loan creation, scoped viewing, borrower-only payment, overdue processing by branch manager or HighAdmin.
Employee management	Hiring, login creation, titles, suspension, termination, temporal branch history, transfers, manager governance.
Authentication	Username/password login, SHA2-256 password comparison, opaque server-side session token.
Authorization	Effective roles, branch scope, ownership checks, database procedure checks, route middleware, SQL Server roles.
SQL injection defense	Parameter binding through <code>request.input</code> ; stored procedures; report identifiers chosen from server whitelist only.
No ORM	Direct <code>mssql</code> driver calls, explicit SQL types, explicit stored-procedure names, and SQL-owned business logic.
Audit/logging	<code>AuditLog</code> , safe audit view, branch ledger, login/logout/expiry and business-operation audit actions.
Session management	Server-side <code>Sessions</code> table with fixed ten-minute <code>ExpiresAt</code> and explicit revocation.

Requirement	Implementation
Responsive interface	Separate role-aware workspaces with responsive CSS, dynamic forms, drop-down catalogues, and mobile layout.
Reporting	Fourteen API reports backed by controlled views, separate SQL pools, paging, newest-first ordering, branch scoping, and CSV export.

Chapter 2

System Architecture

2.1 Layered architecture

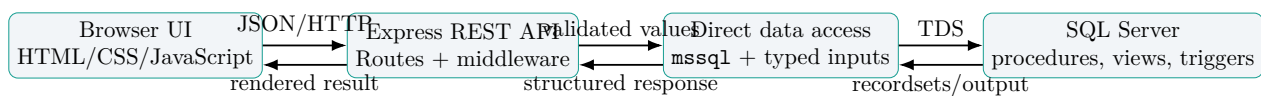


Figure 2.1: End-to-end architecture

2.2 Responsibility boundaries

- **Frontend:** presentation, user input, client-side convenience validation, token storage, calling APIs, rendering role-specific sections, formatting dates, and exporting visible report data.
- **Backend:** HTTP routing, required-field validation, bearer-token extraction, application-role checks, branch/ownership prechecks, SQL type binding, response normalization, CORS, rate limiting, and error translation.
- **Database:** source of truth for identity, effective roles, business rules, referential integrity, transactional consistency, locking, balance changes, installment state, audit records, reporting views, and scheduled maintenance.

2.3 Ports and CORS

The frontend static server normally listens on 127.0.0.1:5500; the backend listens on 127.0.0.1:4000; SQL Server normally listens on port 1433. Different ports are different browser origins, so the backend allows only configured frontend origins through `CORS_ORIGIN`. CORS is a browser boundary and is not a substitute for authentication or authorization.

Chapter 3

Database Conceptual and Relational Design

3.1 Core banking ER model

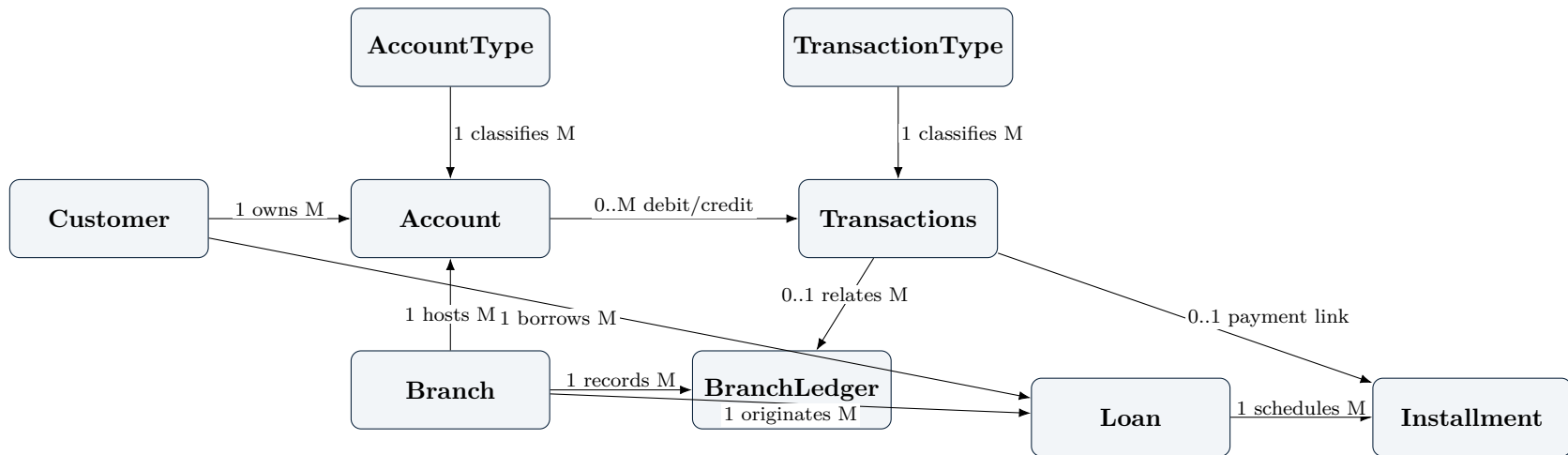


Figure 3.1: Core banking relationships

3.2 Identity, role, and workforce ER model

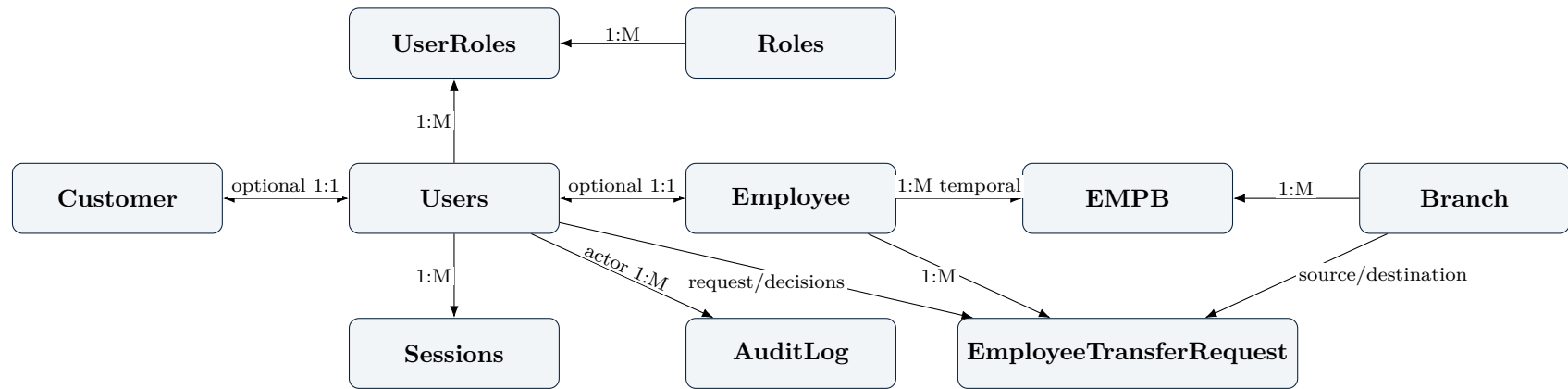


Figure 3.2: Application identity, roles, workforce, and transfer workflow

3.3 Normalization to third normal form

- Repeating descriptive values are moved to lookup tables: **AccountType**, **TransactionType**, and **Roles**.
- Many-to-many relationships are decomposed into associative tables: **UserRoles** and temporal **EMPB**.
- Customer, employee, user, account, loan, and branch facts are stored once under their own keys, avoiding update anomalies.
- Loan installments are separate from **Loan** because each loan has a repeating schedule.
- Transaction debit and credit references are foreign keys rather than embedded account data.
- **Branch.Balance** and **BranchLedger** are intentional controlled denormalizations: the former is a cached aggregate and the latter is an audit history. Their consistency is centralized in a trigger.

Chapter 4

Complete Data Dictionary

The following data dictionary is based on the current database schema used by the application.

4.1 Customer

Bank customer master data; soft-deleted through IsActive.

Column	Type	Constraint/key	Meaning
CustomerID	INT IDENTITY	NOT NULL, PK	Surrogate customer identifier.
FirstName	NVARCHAR(50)	NOT NULL	Given name.
LastName	NVARCHAR(50)	NOT NULL	Family name.
NationalID	NVARCHAR(20)	NOT NULL, UNIQUE, CHECK	Exactly 10 numeric digits.
BirthDate	DATE	NOT NULL	Date of birth; customer-creation procedures enforce adulthood.
Phone	NVARCHAR(20)	NOT NULL, UNIQUE	Primary telephone number.
Email	NVARCHAR(100)	NOT NULL, UNIQUE	Primary email address.
Address	NVARCHAR(200)	NULL	Postal address.
RegistrationDate	DATE	NOT NULL, DEFAULT today	Registration date.
IsActive	BIT	NOT NULL, DEFAULT 1	Soft-deletion / active-profile flag.

4.2 Branch

Bank branch master data and a trigger-maintained aggregate balance.

Column	Type	Constraint/key	Meaning
BranchID	INT IDENTITY	NOT NULL, PK	Surrogate branch identifier.
BranchName	NVARCHAR(100)	NOT NULL	Human-readable branch name.
BranchCode	NVARCHAR(20)	NOT NULL, UNIQUE	Short operational branch code.
Balance	DECIMAL(18,2)	NOT NULL, DEFAULT 0	Aggregate of account-balance changes, maintained by trigger.
City	NVARCHAR(50)	NOT NULL	Branch city.
Address	NVARCHAR(200)	NULL	Branch address.
Phone	NVARCHAR(20)	NULL	Branch phone.

4.3 AccountType

Lookup for account products and product rules.

Column	Type	Constraint/key	Meaning
AccountTypeID	INT IDENTITY	NOT NULL, PK	Account-product identifier.
TypeName	NVARCHAR(50)	NOT NULL, UNIQUE	Product name.
Description	NVARCHAR(200)	NULL	Product description.
MinBalance	DECIMAL(18,2)	NOT NULL, DEFAULT 0	Minimum allowed balance.
InterestRate	DECIMAL(5,2)	NOT NULL, DEFAULT 0	Monthly interest percentage used by maintenance procedure.
MonthlyFee	DECIMAL(18,2)	NOT NULL, DEFAULT 0	Configured product fee (present in schema for policy use).

4.4 Account

Customer account at one branch and of one account type.

Column	Type	Constraint/key	Meaning
AccountID	INT IDENTITY	NOT NULL, PK	Surrogate account identifier.
AccountNumber	NVARCHAR(30)	NOT NULL, UNIQUE, CHECK	Exactly 12 numeric digits; generated from sequence.
CustomerID	INT	NOT NULL, FK -> Customer	Account owner.
BranchID	INT	NOT NULL, FK -> Branch	Owning branch.
AccountTypeID	INT	NOT NULL, FK -> AccountType	Product type.
Balance	DECIMAL(18,2)	NOT NULL, DEFAULT 0	Posted balance; pending outgoing amounts are subtracted separately for available balance.
OpenDate	DATE	NOT NULL	Opening date.
CloseDate	DATE	NULL, CHECK	Must be null or on/after OpenDate.
AccountStatus	NVARCHAR(20)	NOT NULL, CHECK	Active, Closed, Frozen, or Dormant.
FrozenPreviousStatus	NVARCHAR(20)	NULL, CHECK	Stores Active/Dormant state for unfreeze restoration.

4.5 Employee

Personnel master data.

Column	Type	Constraint/key	Meaning
EmployeeID	INT IDENTITY	NOT NULL, PK	Surrogate employee identifier.
NationalID	NVARCHAR(20)	NOT NULL, UNIQUE	Employee national identifier.
FirstName	NVARCHAR(50)	NOT NULL	Given name.
LastName	NVARCHAR(50)	NOT NULL	Family name.
HireDate	DATE	NOT NULL	Hire date.

Column	Type	Constraint/key	Meaning
JobTitle	NVARCHAR(100)	NOT NULL	Operational title; manager titles are special.
Salary	DECIMAL(18,2)	NOT NULL	Salary.
Phone	NVARCHAR(20)	NULL	Employee phone.
Email	NVARCHAR(100)	NULL	Employee email.
EmpStatus	NVARCHAR(20)	NOT NULL, CHECK	Active, OnLeave, or Terminated.
CanAccessAdmin	BIT	NOT NULL, DEFAULT 0	Additional gate for effective Admin privilege.

4.6 EMPB

Temporal employee-to-branch assignment bridge.

Column	Type	Constraint/key	Meaning
EMPBID	INT IDENTITY	NOT NULL, PK	Assignment identifier.
EmployeeID	INT	NOT NULL, FK -> Employee	Assigned employee.
BranchID	INT	NOT NULL, FK -> Branch	Assigned branch.
StartDate	DATE	NOT NULL	Assignment start.
EndDate	DATE	NULL, CHECK	Null while current; otherwise not earlier than StartDate.
WorkingStatus	NVARCHAR(20)	NOT NULL, CHECK	Working, Transferred, or Ended.

4.7 TransactionType

Lookup for transaction semantics.

Column	Type	Constraint/key	Meaning
TransactionTypeID	INT IDENTITY	NOT NULL, PK	Transaction-type identifier.
TypeName	NVARCHAR(50)	NOT NULL, UNIQUE	Deposit, Withdrawal, Transfer, or Interest.

4.8 Transactions

Financial transaction queue and history.

Column	Type	Constraint/key	Meaning
TransactionID	INT IDENTITY	NOT NULL, PK	Transaction identifier.
TransactionTypeID	INT	NOT NULL, FK -> TransactionType	Transaction semantic type.
FromAccountID	INT	NULL, FK -> Account	Debit account, where applicable.
ToAccountID	INT	NULL, FK -> Account	Credit account, where applicable.
EmployeeID	INT	NULL, FK -> Employee	Optional staff metadata; must match authenticated employee when supplied.

Column	Type	Constraint/key	Meaning
Amount	DECIMAL(18,2)	NOT NULL, CHECK > 0	Monetary amount.
TransactionDate	DATETIME	NOT NULL, DEFAULT current time	Creation timestamp.
TransactionStatus	NVARCHAR(20)	NOT NULL, CHECK	Pending, Completed, Failed, or Cancelled.
ReadyToCompleteAt	DATETIME	NULL	Earliest finalization time calculated from amount.
CompletedAt	DATETIME	NULL, consistency CHECK	Required only when status is Completed.
Description	NVARCHAR(200)	NULL	Narrative description.

4.9 Loan

Loan contract for a customer and branch.

Column	Type	Constraint/key	Meaning
LoanID	INT IDENTITY	NOT NULL, PK	Loan identifier.
CustomerID	INT	NOT NULL, FK -> Customer	Borrower.
BranchID	INT	NOT NULL, FK -> Branch	Originating branch.
LoanAmount	DECIMAL(18,2)	NOT NULL, CHECK > 0	Principal.
InterestRate	DECIMAL(5,2)	NOT NULL	Loan interest rate; procedure rejects negative rates.
StartDate	DATE	NOT NULL	Loan start.
EndDate	DATE	NULL, CHECK	Null or on/after StartDate.
LoanStatus	NVARCHAR(20)	NOT NULL, CHECK	Active, Paid, or Defaulted.

4.10 Installment

Scheduled loan payment and its transaction link.

Column	Type	Constraint/key	Meaning
InstallmentID	INT IDENTITY	NOT NULL, PK	Installment identifier.
LoanID	INT	NOT NULL, FK -> Loan	Parent loan.
DueDate	DATE	NOT NULL	Due date.
Amount	DECIMAL(18,2)	NOT NULL, CHECK > 0	Installment amount.
PaidDate	DATE	NULL	Date payment transaction completed.
InstallmentStatus	NVARCHAR(20)	NOT NULL, CHECK	Pending, Paid, Late, or Defaulted.
PaymentTransactionID	INT	NULL, FK -> Transactions	Pending/completed payment transaction.

4.11 BranchLedger

Append-style record of branch balance deltas.

Column	Type	Constraint/key	Meaning
BranchLedgerID	INT IDENTITY	NOT NULL, PK	Ledger row identifier.
BranchID	INT	NOT NULL, FK -> Branch	Affected branch.
TransactionID	INT	NULL, FK -> Transactions	Optional related transaction.
DeltaAmount	DECIMAL(18,2)	NOT NULL	Branch-balance delta.
BalanceAfter	DECIMAL(18,2)	NOT NULL	Branch balance after the update batch.
EntryDate	DATETIME	NOT NULL, DEFAULT current time	Ledger timestamp.
Description	NVARCHAR(200)	NULL	Ledger narrative.

4.12 Roles

Application role catalogue.

Column	Type	Constraint/key	Meaning
RoleID	INT IDENTITY	NOT NULL, PK	Role identifier.
RoleName	NVARCHAR(50)	NOT NULL, UNIQUE	Customer, Employee, Admin, or HighAdmin.

4.13 Users

Application login identity linked to customer and optionally employee.

Column	Type	Constraint/key	Meaning
UserID	INT IDENTITY	NOT NULL, PK	Application user identifier.
Username	NVARCHAR(50)	NOT NULL, UNIQUE	Login name.
PasswordHash	CHAR(64)	NOT NULL	Uppercase hexadecimal SHA2-256 hash.
EmployeeID	INT	NULL, FK -> Employee, filtered UNIQUE	Optional one-to-one employee identity.
CustomerID	INT	NULL, FK -> Customer, filtered UNIQUE	Required by effective login model; one-to-one customer identity.
IsActive	BIT	NOT NULL, DEFAULT 1	Application login enabled flag.

4.14 UserRoles

Many-to-many assignment of application users to roles.

Column	Type	Constraint/key	Meaning
UserRoleID	INT IDENTITY	NOT NULL, PK	Assignment identifier.

Column	Type	Constraint/key	Meaning
UserID	INT	NOT NULL, FK -> Users, ON DELETE CASCADE	User.
RoleID	INT	NOT NULL, FK -> Roles, ON DELETE CASCADE	Assigned role.
UQ(UserID,RoleID)	Constraint	UNIQUE	Prevents duplicate assignments.

4.15 Sessions

Server-side bearer-token sessions.

Column	Type	Constraint/key	Meaning
SessionID	INT IDENTITY	NOT NULL, PK	Session identifier.
UserID	INT	NOT NULL, FK -> Users, ON DELETE CASCADE	Authenticated user.
SessionToken	NVARCHAR(200)	NOT NULL, UNIQUE	Opaque token composed from two GUIDs.
LoginTime	DATETIME	NOT NULL, DEFAULT current time	Login timestamp.
ExpiresAt	DATETIME	NOT NULL, DEFAULT +10 min	Fixed absolute expiration.
LogoutTime	DATETIME	NULL	Explicit logout or expiration timestamp.
IsActive	BIT	NOT NULL, DEFAULT 1	Server-side revocation flag.

4.16 AuditLog

Application and database activity audit record.

Column	Type	Constraint/key	Meaning
AuditID	INT IDENTITY	NOT NULL, PK	Audit identifier.
UserID	INT	NULL, FK -> Users	Actor, or null for system jobs.
ActionType	NVARCHAR(100)	NOT NULL	Structured action label.
TableName	NVARCHAR(100)	NULL	Logical object affected.
RecordID	INT	NULL	Logical record identifier.
ActionDate	DATETIME	NOT NULL, DEFAULT current time	Audit timestamp.
Details	NVARCHAR(MAX)	NULL	Context for the audited action; the safe reporting view omits password hashes and session tokens.

4.17 EmployeeTransferRequest

Two-manager employee transfer workflow.

Column	Type	Constraint/key	Meaning
TransferRequestID	INT IDENTITY	NOT NULL, PK	Request identifier.
EmployeeID	INT	NOT NULL, FK -> Employee	Employee being transferred.
FromBranchID	INT	NOT NULL, FK -> Branch	Current branch.
ToBranchID	INT	NOT NULL, FK -> Branch, CHECK different	Destination branch.
RequestedByUserID	INT	NOT NULL, FK -> Users	Requester.
RequestType	NVARCHAR(30)	NOT NULL, CHECK	EmployeeRequest or ManagerRequest.
Reason	NVARCHAR(500)	NULL	Request reason.
CurrentManagerUserID	INT	NULL, FK -> Users	Current-branch approver.
CurrentManagerDecision	NVARCHAR(20)	NULL, CHECK	Approved or Rejected.
CurrentManagerDecisionDate	DATETIME	NULL	Decision time.
CurrentManagerNote	NVARCHAR(500)	NULL	Decision note.
DestinationManagerUserID	INT	NULL, FK -> Users	Destination approver.
DestinationManagerDecision	NVARCHAR(20)	NULL, CHECK	Approved or Rejected.
DestinationManagerDecisionDate	DATETIME	NULL	Decision time.
DestinationManagerNote	NVARCHAR(500)	NULL	Decision note.
TransferStatus	NVARCHAR(50)	NOT NULL, CHECK	Pending stages, rejected stages, Completed, or Cancelled.
EffectiveDate	DATE	NULL	Transfer effective date.
CreatedAt	DATETIME	NOT NULL, DEFAULT current time	Creation timestamp.
CompletedAt	DATETIME	NULL	Completion timestamp.
CancelledAt	DATETIME	NULL	Cancellation timestamp.

Chapter 5

Relationships, Keys, Constraints, and Indexes

5.1 Cardinality summary

Parent	Child	Meaning
Customer	Account	One customer can own many accounts; each account has exactly one owner.
Branch	Account	One branch hosts many accounts; each account belongs to one branch.
AccountType	Account	One product type classifies many accounts.
Employee	EMPB	One employee has many historical assignments but only one filtered current Working assignment.
Branch	EMPB	One branch has many current/historical employee assignments.
TransactionType	Transactions	One type classifies many transactions.
Account	Transactions	An account may appear as source and/or destination across many transactions.
Customer	Loan	One customer may have many loans.
Branch	Loan	One branch may originate many loans.
Loan	Installment	One loan owns many installments.
Transactions	Installment	An installment may refer to a payment transaction; synchronization is procedure/trigger controlled.
Users	UserRoles	One user has many role assignments.
Roles	UserRoles	One role is assigned to many users.
Users	Sessions	One user may have many sessions.
Users	AuditLog	One user may produce many audit records; system jobs may use NULL.
Employee/Branch/ Users	EmployeeTransfer Request	Request links the employee, source and destination branches, requester, and both manager decisions.

5.2 Important indexes and why they exist

- `IX_Account_CustomerStatus` and `IX_Account_BranchStatus`: common customer/branch account lists and status filtering.
- `IX_Transactions_FromAccount` and `IX_Transactions_ToAccount`: newest transaction history from either direction.
- Filtered `IX_Employee_OneActiveBranch`: physically prevents more than one current assignment

per employee.

- `IX_EMPB_CurrentBranchEmployees`: current branch workforce lookup.
- `IX_Sessions_ActiveExpiry`: token validation/cleanup by active state and expiration.
- `IX_AuditLog_UserDate` and `IX_AuditLog_TableRecord`: newest audit history and object lookup.
- Filtered unique transfer index: one open transfer request per employee.
- Filtered unique `Users.CustomerID` and `Users.EmployeeID`: one application login per identity.

5.3 Account number generation

`dbo.seq_AccountNumber` provides concurrency-safe numeric values. `sp_Account_Open` formats the value as a 12-digit account number and rejects sequence overflow. This avoids unsafe patterns such as `MAX(AccountNumber)+1`.

Chapter 6

Database Business Rules and Integrity Automation

6.1 Account lifecycle

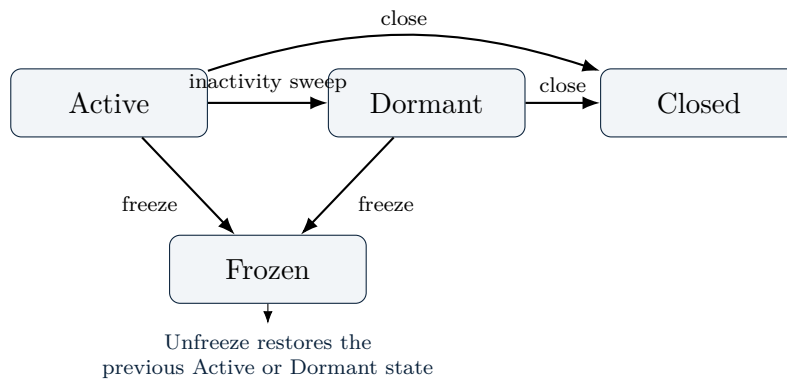


Figure 6.1: Account status transitions

Financial transactions require ownership. Withdrawal/transfer source accounts must be Active; deposits reject Closed/Frozen destinations; account closing is blocked for Frozen accounts and for accounts referenced by Pending transactions.

6.2 Available balance and pending holds

Posted `Account.Balance` is not the same as available balance. The helper view sums Pending outgoing transactions:

$$\text{AvailableBalance} = \text{Balance} - \sum \text{PendingOutgoingAmount}.$$

Minimum-balance checks use available balance so a customer cannot overspend by initiating multiple Pending withdrawals/transfers before finalization.

6.3 Transaction amount-dependent delay

Amount	Completion delay
< 5,000	0 seconds
< 25,000	60 seconds
< 100,000	180 seconds
≥ 100,000	300 seconds

`ReadyToCompleteAt` is therefore a scheduled processing time, not a random timestamp.

6.4 Triggers

6.4.1 trg_EMPB_PreventDateOverlap

Table: EMPB. Rejects overlapping branch-assignment date ranges for one employee.

6.4.2 TR_Account_PreventCloseWithPending

Table: Account. Rejects any transition to Closed while a Pending transaction references the account.

6.4.3 TR_Account_SyncBranchLedger

Table: Account. On balance insert/update, updates Branch.Balance and appends BranchLedger deltas.

6.4.4 TR_Customer_PreventDeactivateWithPending

Table: Customer. Rejects deactivation when any customer account has a Pending transaction.

6.4.5 TR_Transactions_SyncInstallmentStatus

Table: Transactions. When payment transaction completes/fails, synchronizes installment and loan status.

6.4.6 TR_Transactions_EnforceTypePattern

Table: Transactions. Enforces From/To account pattern for Deposit, Interest, Withdrawal, and Transfer.

6.5 Effective-role function

`dbo.fn_UserHasEffectiveRole(UserID,RoleName)` does not merely check `UserRoles`. It requires an active user and active Customer profile. Employee requires an active linked employee. Admin additionally requires Employee role, active employee, `CanAccessAdmin=1`, and job title Branch Manager or Vice Manager. HighAdmin requires no linked EmployeeID and must not be combined with Employee/Admin.

6.6 Transactional consistency and concurrency

Critical procedures enable `SET XACT_ABORT ON`, use `TRY/CATCH`, and wrap write operations in explicit transactions. Before rechecking balances or workflow state, they use locks such as `UPDLOCK` and `HOLDLOCK`. These measures reduce race conditions during balance updates, account-state changes, session logout, transfer approval, and manager replacement.

Chapter 7

Stored Procedures

7.1 Procedure design pattern

Most write procedures follow this sequence:

1. Validate required inputs and enumerated values.
2. Resolve the caller and recalculate effective privilege.
3. Resolve ownership or current branch.
4. Begin an atomic transaction and lock target rows where necessary.
5. Recheck business state under lock.
6. Perform inserts/updates.
7. Insert an audit record.
8. Commit and return recordsets/output parameters; otherwise roll back and raise a structured SQL error.

7.2 Authentication and session

Procedure	Primary caller	Purpose and key rules
sp_User_SignUp	Public	Creates/reuses a Customer through the customer procedure, creates Users row, hashes password, assigns Customer role, audits signup.
sp_User_Login	Public	Validates one login identifier plus password, recalculates effective roles, creates a fixed-lifetime session token, returns identity/roles/ExpiresAt, audits login.
sp_User_ValidateSession	Every protected request	Validates token, active state and expiry; recalculates effective Customer/Employee/Admin/HighAdmin roles and current branch; returns session context.
sp_User_Logout	Authenticated	Marks the session inactive, writes Logout-Time, and audits logout.
sp_HighAdmin_CreateInitial User	One-time public setup	Creates the first HighAdmin only when none exists; links an active Customer identity, assigns Customer+HighAdmin, and audits creation.

7.3 Customer

Procedure	Primary caller	Purpose and key rules
sp_Customer_Create	Employee/Admin/ HighAdmin	Validates identity/contact uniqueness and age; inserts customer atomically and audits CustomerCreated.
sp_Customer_Search	Employee/Admin/ HighAdmin	Branch-scopes staff through customer-account relationships; HighAdmin sees all; supports name/NationalID/phone/email filters.
sp_Customer_GetByID	Authenticated accord- ing to role	Returns own customer to Customer; current-branch customer to staff; any cus- tomer to HighAdmin.
sp_Customer_Update	Customer self or scoped staff	Allows own profile or branch-scoped staff update; checks uniqueness and audits.
sp_Customer_Delete	Scoped staff/ HighAdmin	Soft-deactivates; blocks pending-transaction cases; branch scopes staff and audits.

7.4 Accounts

Procedure	Primary caller	Purpose and key rules
sp_AccountType_ListOptions	Authenticated	Returns account-product options for con- trolled dropdowns.
sp_Account_Open	Customer self-service	Generates 12-digit account number, vali- dates branch/product/minimum deposit, inserts account and optional pending initial deposit, audits.
sp_Account_Search	Authenticated	Customer own accounts; Employee/Admin current branch; HighAdmin all. Supports rich filters and calculates available balance.
sp_Account_GetInfo	Authenticated	Returns account details under the same ownership/branch/all visibility model and audits view.
sp_Transaction_GetAccountH istory	Authenticated	Returns newest transaction history for an authorized account and date range.
sp_Account_GetAvailableBal ance	Authenticated	Returns posted balance minus pending outgoing holds.
sp_Account_Freeze	Employee/Admin/ HighAdmin within scope	Stores previous Active/Dormant state and sets Frozen.
sp_Account_Unfreeze	Admin/HighAdmin within scope	Restores FrozenPreviousStatus.
sp_Account_Close	Employee/Admin/ HighAdmin within scope	Blocks already closed, frozen, or pending- transaction accounts; sets close date and audits.
sp_Account_ChangeType	Employee/Admin/ HighAdmin within scope	Requires Active/Dormant, no pending transaction, sufficient available balance for target type; audits.
sp_Account_DormantSweep	Admin/HighAdmin or job	Marks qualifying inactive accounts Dor- mant.

Procedure	Primary caller	Purpose and key rules
sp_Account_ApplyMonthlyInterest	Admin/HighAdmin or job	Creates interest transactions for eligible account types atomically.

7.5 Transactions

Procedure	Primary caller	Purpose and key rules
sp_Transaction_Deposit	Account owner	Validates destination ownership and account state; inserts Pending deposit with scheduled completion.
sp_Transaction-Withdrawal	Account owner	Validates source ownership, Active state, minimum/available balance; inserts Pending withdrawal.
sp_Transaction_Transfer	Source-account owner	Validates both accounts, ownership, status, available funds, and inserts Pending transfer.
sp_Transaction_Finalize	Owner or system batch	Locks transaction/accounts, waits until ReadyToCompleteAt, posts balance changes, marks Completed/Failed, audits.
sp_Transaction_ProcessPendingBatch	Admin/HighAdmin or SQL Agent	Set-based finalization of all ready Pending transactions.
sp_Transaction_Reverse	Transaction owner	Cancels Pending or reverses Completed transaction while protecting minimum balance and linked installment semantics.

7.6 Loans and installments

Procedure	Primary caller	Purpose and key rules
sp_Loan_Create	Employee/Admin/HighAdmin	Creates a branch loan and installment schedule atomically; validates active customer, branch, amount, rate, and installment count.
sp_Loan_GetStatus	Authorized viewer	Returns loan summary plus installment recordset; customer ownership is enforced in SQL and staff scope is prevalidated by backend.
sp_Loan_PayInstallment	Borrower only	Requires borrower-owned payment account, Active loan, unpaid/non-defaulted installment, and no existing pending payment; creates withdrawal transaction.
sp_Loan_ProcessOverdueInstallments	Branch manager/Admin, HighAdmin, or job	Marks overdue installments Late/Defaulted and loans Defaulted according to branch scope; manual managers only current branch.

7.7 Branches, employees, and transfers

Procedure	Primary caller	Purpose and key rules
sp_Branch_ListOptions	Authenticated	Returns non-sensitive BranchID/code/name/city for dropdowns.
sp_Branch_GetInfo	Employee/Admin/HighAdmin	Normal employee sees current branch; managers current branch; HighAdmin all/selected.
sp_Employee_Hire	Current-branch Admin	Creates ordinary employee and current EMPB assignment in manager branch; no login yet.
sp_Employee_CreateUserAccount	Scoped Admin/HighAdmin	Creates/reuses customer identity and user, links employee, and assigns Customer+Employee.
sp_Employee_Search	Employee/Admin/HighAdmin	Employee sees self; manager current branch; HighAdmin all/history.
sp_Employee_GetInfo	Employee/Admin/HighAdmin	Same target visibility; returns employee and assignment details.
sp_EMPBranchHistory_Get	Employee/Admin/HighAdmin	Self/current-branch/all history, with Vice Manager protection for Branch Manager records.
sp_Employee_ChangeJobTitle	Scoped Admin/HighAdmin	Changes ordinary title; manager-level governance remains HighAdmin-specific.
sp_Employee_Suspend	Scoped Admin/HighAdmin	Sets OnLeave and removes effective Employee/Admin privilege while preserving Customer access.
sp_Employee_Fire	Scoped Admin/HighAdmin	Sets Terminated, closes current assignment, preserves customer login but removes effective staff privilege.
sp_EmployeeTransfer_RequestByEmployee	Active Employee/Admin	Creates open transfer request to a different branch.
sp_EmployeeTransfer_RequestByManager	Admin/HighAdmin	Creates transfer request for an eligible employee.
sp_EmployeeTransfer_ApproveCurrentManager	Current-branch Admin	Approves/rejects first stage only for source branch.
sp_EmployeeTransfer_ApproveDestinationManager	Destination Admin	Approves/rejects second stage; on approval closes old EMPB and creates new current assignment.

7.8 HighAdmin manager governance

Procedure	Primary caller	Purpose and key rules
sp_HighAdmin_HireManager	HighAdmin	Requires existing BranchID and manager title; atomically creates/reuses Customer, Employee, EMPB, Users, and Customer+Employee+Admin roles.
sp_HighAdmin_PromoteToManager	HighAdmin	Promotes active employee with login to Branch Manager/Vice Manager in target branch.
sp_HighAdmin_DowngradeManager	HighAdmin	Changes manager to ordinary title and removes effective Admin capability.

Procedure	Primary caller	Purpose and key rules
sp_HighAdmin_SuspendManager	HighAdmin	Suspends manager-level employee.
sp_HighAdmin_FireManager	HighAdmin	Terminates manager-level employee.
sp_HighAdmin_ReplaceBranchManager	HighAdmin	Atomically downgrades old manager, assigns/promotes replacement, and enforces one active Branch Manager.

7.9 Critical transaction flow

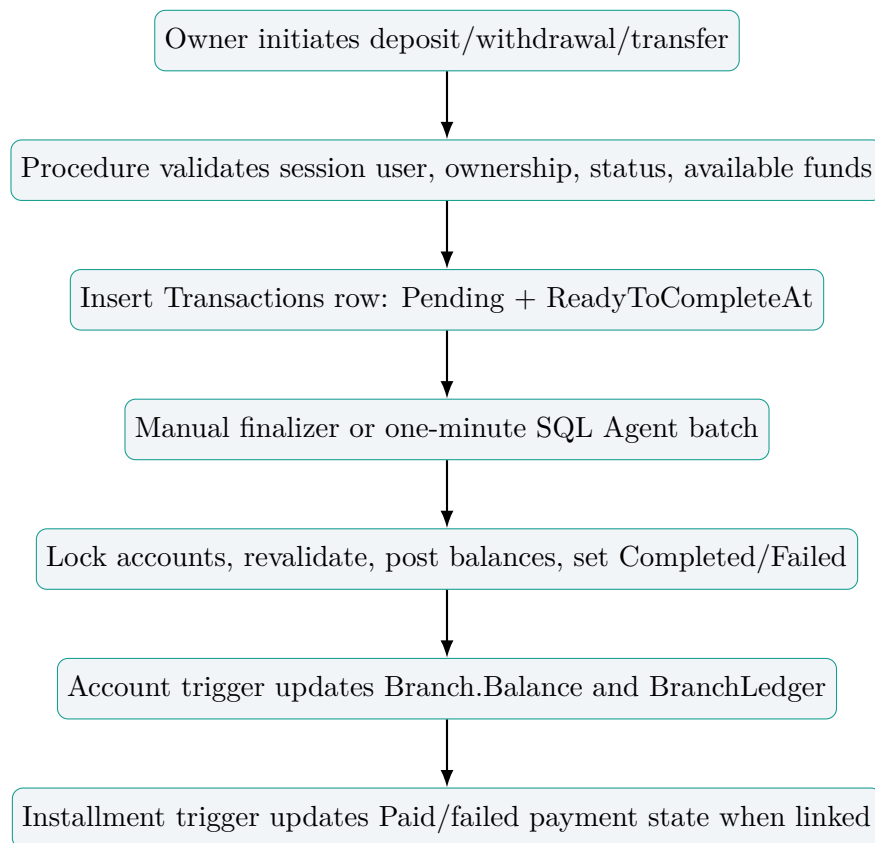


Figure 7.1: Financial transaction lifecycle

Chapter 8

Views, Reports, and Scheduled Jobs

8.1 Reporting views

View	Purpose
vw_AccountPendingOutgoing	Helper aggregate of Pending outgoing amounts by source account.
vw_CustomerBasicProfile	Reporting-safe customer profile with masked NationalID and computed age.
vw_CustomerAccountSummary	Customer/account/branch/type summary with posted, pending, and available balances.
vw_AccountOperationalSummary	Operational account directory with masked customer ID and available balance.
vw_EmployeeDirectory	Current employee directory; excludes salary and masks NationalID.
vw_HighAdmin_EmployeeBranchOverview	Full current and historical employee/branch assignments including salary.
vw_PendingTransactions	Pending transaction work queue.
vw_LoanOperationalSummary	Loan summary with installment counts and remaining amount.
vw_HighAdmin_BranchFinancialOverview	Branch aggregate reconciliation, account totals, and staffing counts.
vw_HighAdmin_UserAccessOverview	User/customer/employee/role overview without password hashes or session tokens.
vw_AuditTrail_Safe	Audit records joined to username.
vw_BranchLedgerReport	Branch ledger joined to branch identity.
vw_DailyTransactionSummary	Daily counts and amounts by type and status.
vw_AccountStatusSummary	Account counts and balances by branch/status.
vw_LoanOverdueSummary	Overdue count, amount, and oldest due date by loan.

8.2 API report catalogue

Report key	Authorized roles and data scope	Default order
customer-basic-profile	Employee/Admin/HighAdmin; staff see customers related to the current branch through account ownership.	RegistrationDate DESC
customer-account-summary	Employee/Admin/HighAdmin; current branch for staff.	OpenDate DESC
account-operational-summary	Employee/Admin/HighAdmin; current branch for staff.	OpenDate DESC

Report key	Authorized roles and data scope	Default order
employee-directory	Employee/Admin/HighAdmin; self or current branch for staff according to role.	HireDate DESC
pending-transactions	Employee/Admin/HighAdmin; accounts linked to the current branch for staff.	TransactionDate DESC
loan-operational-summary	Employee/Admin/HighAdmin; current branch for staff.	StartDate DESC
daily-transaction-summary	Admin/HighAdmin; role-protected aggregate.	TransactionDay DESC
account-status-summary	Admin/HighAdmin; current branch for Admin, all for HighAdmin.	BranchID DESC
loan-overdue-summary	Admin/HighAdmin; current branch for Admin, all for HighAdmin.	Oldest overdue date DESC
highadmin-employee-branch-overview	HighAdmin only; all current and historical assignments.	StartDate DESC
highadmin-branch-financial-overview	HighAdmin only; all branches.	BranchID DESC
highadmin-user-access-overview	HighAdmin only; all application users and effective access context.	UserID DESC
audit-trail	HighAdmin only; all rows through the audit SQL pool.	ActionDate DESC
branch-ledger	HighAdmin only; all rows through the audit SQL pool.	EntryDate DESC

The backend chooses the view name and ORDER BY expression from a hard-coded map. The client cannot submit a view name or SQL identifier. Only paging and branch values are parameters. This is why the small dynamic report query remains safe from SQL injection.

8.3 Database pools and least privilege

The backend has four connection pools:

- **app**: normal procedure execution and limited operational views.
- **report**: ordinary reporting views.
- **audit**: audit trail and branch ledger views.
- **highadminReport**: HighAdmin-only reporting views.

SQL Server database roles are `dbrole_bank_app_executor`, `dbrole_bank_reporting`, `dbrole_bank_auditor`, `dbrole_bank_highadmin_viewer`, and `dbrole_bank_maintenance`. Direct DML permissions on base tables are revoked from application/report personas; access is through procedure execution and selected views.

8.4 SQL Server Agent jobs

Job and schedule	Command	Purpose
Process Pending Bank Transactions Every minute	EXEC dbo.sp_Transaction_ProcessPendingBatch;	Completes all Pending transactions whose ReadyToCompleteAt has arrived.
Process Overdue Loan Installments Daily at 00:00	EXEC dbo.sp_Loan_ProcessOverdueInstallments;	Marks late/defaulted installments and related loans across all branches.

Job and schedule	Command	Purpose
Apply Monthly Account Interest Monthly, day 1 at 00:05	EXEC dbo.sp_Account_ApplyMonthlyInterest;	Creates eligible interest-credit transactions.

Chapter 9

Authentication, Authorization, Sessions, and Audit

9.1 Login and session sequence

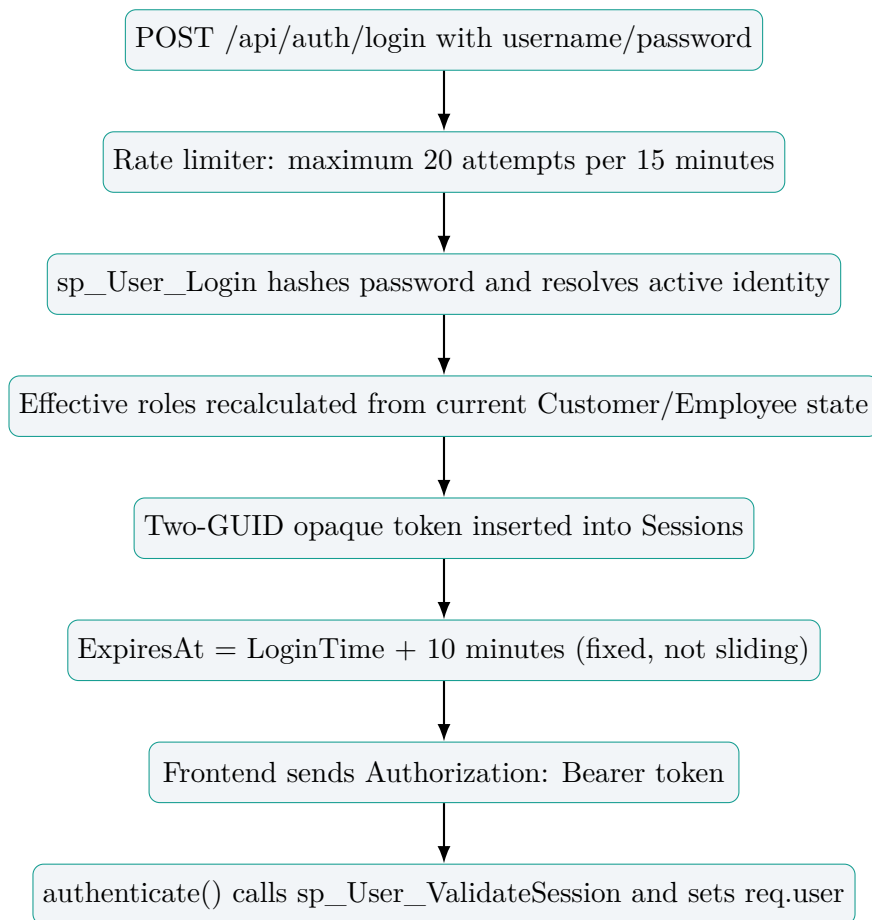


Figure 9.1: Authentication and session validation

9.2 How req.user is created

Listing 9.1: Authentication middleware concept

```
1 const token = extractBearerToken(req);
2 const result = await executeProcedure('dbo.sp_User_ValidateSession', {
3   inputs: { SessionToken: TYPES.token, RequiredRole: TYPES.string50 },
4   values: { SessionToken: token, RequiredRole: requiredRole }
5 });
6 const user = result.recordset[0];
```

```

7 req.sessionToken = token;
8 req.user = { ...user, roles: parseRoles(user.EffectiveRoles) };
9 next();

```

`req.user.UserID` is therefore a custom Express request property populated from a validated SQL result. It is not supplied by the browser and is unrelated to ORM behavior.

9.3 Assigned roles versus effective roles

Role	Effective-role conditions
Customer	Active Users row, active linked Customer, and assigned Customer role.
Employee	Customer conditions plus linked Employee, assigned Employee role, and <code>Employee.EmpStatus=Active</code> .
Admin	Effective Employee plus assigned Admin role, <code>CanAccessAdmin=1</code> , and title Branch Manager or Vice Manager.
HighAdmin	Active Customer identity, assigned HighAdmin, no EmployeeID, and no Employee/Admin role combination.

This design means OnLeave/Terminated staff can retain personal Customer access while losing employee/admin privilege immediately, even with an existing token.

9.4 Branch and ownership authorization

- Customer and `scope=mine`: CustomerID from validated session.
- Employee/Admin operational scope: CurrentBranchID from the current EMPB assignment.
- HighAdmin operational scope: all branches unless a branch is explicitly selected for a supported operation.
- Transactions: owner-only regardless of Employee/Admin/HighAdmin roles.
- Installment payment: borrower-only, and the payment account must belong to the same borrower.
- Managers: employees and customers only from their current branch; customer-to-branch relation is derived through Account.

9.5 Auditing

Audit entries are inserted by procedures for login, logout, expiry, signup, searches, views, customer/account/transaction/loan/employee/manager operations, transfers, and maintenance. HighAdmin reads `vw_AuditTrail_Safe`; raw password hashes and session tokens are not exposed by reporting views. Branch balance changes are separately represented in `BranchLedger`.

Chapter 10

SQL Injection Control and Direct SQL Data Access

10.1 Evidence that the project does not use an ORM

The backend accesses SQL Server directly through the `mssql` package. In this project, `mssql` is used as a database driver: it opens SQL Server connection pools, binds typed parameters, executes named stored procedures or controlled queries, and returns recordsets. It does not map database tables to JavaScript classes.

The conclusion that no ORM is used is supported by the repository itself:

Evidence	What is present in this project
Dependencies	<code>package.json</code> contains <code>mssql</code> , <code>Express</code> , <code>CORS</code> , <code>Helmet</code> , <code>Morgan</code> , <code>rate limiting</code> , and <code>environment</code> support. It does not contain <code>Sequelize</code> , <code>TypeORM</code> , <code>Prisma</code> , <code>Knex</code> , <code>Objection</code> , <code>Bookshelf</code> , <code>MikroORM</code> , or <code>Drizzle</code> .
Database connection	<code>src/config/db.js</code> imports <code>mssql</code> and creates <code>sql.ConnectionPool</code> objects directly.
Data-access helper	<code>src/utils/procedure.js</code> calls <code>request.input(...)</code> and then <code>request.execute(procedureName)</code> or <code>request.query(queryText)</code> .
Routes	Route files name the SQL procedure explicitly and define each SQL type explicitly. For example, the account-opening route calls <code>dbo.sp_Account_Open</code> and binds <code>UserID</code> , <code>BranchID</code> , <code>AccountTypeID</code> , and <code>InitialDeposit</code> .
Project structure	There is no ORM model or entity layer, no <code>schema.prisma</code> , no generated migration folder, and no repository pattern built on ORM entities.
Business logic	Balance checks, account ownership, loan rules, branch scope, audit insertion, locks, transactions, and state transitions are implemented in T-SQL procedures, triggers, constraints, and functions.
Returned data	The backend reads SQL <code>recordset</code> , <code>recordsets</code> , output parameters, and <code>rowsAffected</code> ; it does not receive mapped entity instances.

Listing 10.1: Direct SQL Server connection pool from `src/config/db.js`

```
1 const sql = require('mssql');
2
3 const pool = new sql.ConnectionPool(config);
4 pools.set(name, pool.connect());
```

Listing 10.2: Typed procedure execution from `src/utils/procedure.js`

```
1 const pool = await getPool(poolName);
2 const request = pool.request();
3
```

```
4 request.input('UserID', sql.Int, values.UserID);
5 request.input('BranchID', sql.Int, values.BranchID);
6
7 const result = await request.execute('dbo.sp_Account_Open');
```

Repository check

Open `package.json`, `src/config/db.js`, `src/utils/procedure.js`, and one route file. The dependency list shows no ORM package; the source code shows a direct `mssql` connection, explicit SQL types, and explicit stored-procedure names. The corresponding T-SQL procedure shows where the business rules are implemented.

10.2 Difference between a driver and an ORM

A SQL driver provides a communication layer between the application and the database. An ORM adds an object model above that layer and normally maps tables to classes, creates queries from method calls, tracks entity changes, and manages relationships through model definitions. None of those ORM services is used here. The application sends parameters to SQL Server and relies on database objects written manually.

10.3 SQL injection controls

1. User-supplied values are passed with explicit SQL types through `request.input`.
2. Stored procedures receive values as parameters; user text is not concatenated into SQL statements.
3. Report queries use view names and ordering expressions selected from a server-owned whitelist.
4. Paging values, branch identifiers, record identifiers, names, dates, amounts, and statuses remain bound parameters.
5. Route validation and SQL-side validation provide two separate checks.
6. Database roles restrict direct access to base tables and grant only the required procedures or views.

Important distinction

Stored procedures are not automatically immune to SQL injection. They are safe only when values remain parameters or when any dynamic identifier is selected from trusted server code. In this project, client input is never used as an arbitrary table name, view name, column name, or SQL fragment.

Chapter 11

Backend Architecture

11.1 Technology and startup

The backend uses Node.js and Express. Supporting packages provide SQL Server access, environment configuration, security headers, CORS, request logging, and rate limiting. `src/server.js` starts the HTTP listener and closes the SQL pools during shutdown. `src/app.js` installs the middleware, health route, module routers, 404 handler, and centralized error handler.

11.2 Configuration and database connection

`src/config/env.js` validates environment variables and supplies defaults. `src/config/db.js` lazily creates named connection pools. SQL dates use `DB_USE_UTC=false` so SQL Server local `DATETIME` values are not incorrectly interpreted as UTC. Pooling avoids opening a new database connection for every request.

11.3 Request handling pattern

1. Express matches the route and parses JSON/query/path parameters.
2. `authenticate()` validates the bearer session for protected routes.
3. `authorize(...roles)` performs broad application-role checks.
4. Route code overrides identity/branch fields with trusted `req.user` values.
5. `executeProcedure` or `executeQuery` binds typed parameters and selects the correct pool.
6. SQL Server performs detailed authorization, transaction, audit, and integrity logic.
7. `ok/created` normalizes DATE vs DATETIME values and returns a consistent JSON envelope.
8. SQL errors are normalized into structured HTTP errors; unknown routes return 404.

11.4 API response shape

```
1 {  
2   "success": true,  
3   "data": [...],  
4   "meta": {  
5     "accessScope": "branch",  
6     "branchID": 2,  
7     "dateFields": { "TransactionDate": "datetime" }  
8   }  
9 }
```

Failures use `success:false` and `error.message`; SQL errors also expose number/state/procedure/-line in development.

11.5 Complete API inventory

The following tables list the implemented HTTP surface. Endpoint paths and SQL object names are typeset with breakable monospaced text so that the inventory remains readable in print.

11.5.1 Setup, authentication, and customers

Method and endpoint	Access	SQL target
POST /api/setup/initial-high-admin	Public, one-time	sp_HighAdmin_CreateInitialUser
POST /api/auth/signup	Public	sp_User_SignUp
POST /api/auth/login	Public, rate-limited	sp_User_Login
POST /api/auth/logout	Authenticated	sp_User_Logout
GET /api/auth/me	Authenticated	Validated session context
GET /api/customers	Employee/Admin: current branch; HighAdmin: all	sp_Customer_Search
POST /api/customers	Employee/Admin/HighAdmin	sp_Customer_Create
PUT /api/customers/:customerID	Branch-scoped staff or High-Admin	sp_Customer_Update
DELETE /api/customers/:customerID	Branch-scoped staff or High-Admin	sp_Customer_Delete

11.5.2 Accounts and transactions

Method and endpoint	Access	SQL target
GET /api/accounts/mine	Authenticated user's own accounts	sp_Account_Search
GET /api/accounts	Customer: own; Employee/Admin: branch; HighAdmin: all	sp_Account_Search
GET /api/accounts/options/account-types	Authenticated safe catalogue	sp_AccountType_ListOptions
GET /api/accounts/:accountID	Authorized owner/branch/all viewer	sp_Account_GetInfo
GET /api/accounts/:accountID/history	Authorized owner/branch/all viewer	sp_Transaction_GetAccountHistory
POST /api/accounts	Customer self-service	sp_Account_Open
POST /api/accounts/:id/freeze	Scoped Employee/Admin/HighAdmin	sp_Account_Freeze
POST /api/accounts/:id/unfreeze	Scoped Admin/HighAdmin	sp_Account_Unfreeze
POST /api/accounts/:id/close	Scoped Employee/Admin/HighAdmin	sp_Account_Close
POST /api/accounts/:id/change-type	Scoped Employee/Admin/HighAdmin	sp_Account_ChangeType
POST /api/transactions/deposit	Account owner only	sp_Transaction_Deposit
POST /api/transactions/withdraw	Account owner only	sp_Transaction_Withdrawal
POST /api/transactions/transfer	Source-account owner only	sp_Transaction_Transfer

Method and endpoint	Access	SQL target
POST /api/transactions/:id/finalize	Transaction/account owner policy	sp_Transaction_Finalize
POST /api/transactions/:id/reverse	Transaction/account owner policy	sp_Transaction_Reverse
POST /api/transactions/maintenance/process-pending-batch	Admin/HighAdmin maintenance	sp_Transaction_ProcessPendingBatch

11.5.3 Loans, employees, and transfers

Method and endpoint	Access	SQL target
GET /api/loans	Customer: own; Employee/Admin: branch; HighAdmin: all	vw_LoanOperationalSummary
POST /api/loans	Employee/Admin current branch; HighAdmin any branch	sp_Loan_Create
GET /api/loans/:id/status	Authorized owner/branch/all viewer	sp_Loan_GetStatus
POST /api/loans/installments/:id/pay	Borrower only, using an owned payment account	sp_Loan_PayInstallment
POST /api/loans/maintenance/process-overdue-installments	Admin current branch; HighAdmin all or selected branch	sp_Loan_ProcessOverdueInstallments
GET /api/employees	Employee: self; Admin: branch; HighAdmin: all	sp_Employee_Search
GET /api/employees/:id	Employee: self; Admin: branch; HighAdmin: all	sp_Employee_GetInfo
POST /api/employees	Admin's current branch, ordinary job title	sp_Employee_Hire
POST /api/employees/:id/create-user-account	Scoped Admin/HighAdmin	sp_Employee_CreateUserAccount
POST /api/employees/:id/change-job-title	Scoped Admin/HighAdmin	sp_Employee_ChangeJobTitle
POST /api/employees/:id/suspend	Scoped Admin/HighAdmin	sp_Employee_Suspend
POST /api/employees/:id/fire	Scoped Admin/HighAdmin	sp_Employee_Fire
GET /api/employees/:id/branch-history	Employee: self; Admin: branch; HighAdmin: all	sp_EMPBranchHistory_Get
POST /api/employee-transfers/request-by-employee	Employee/Admin	sp_EmployeeTransfer_RequestByEmployee
POST /api/employee-transfers/request-by-manager	Admin/HighAdmin	sp_EmployeeTransfer_RequestByManager
POST /api/employee-transfers/:id/current-manager-decision	Admin of source branch	sp_EmployeeTransfer_ApproveCurrentManager
POST /api/employee-transfers/:id/destination-manager-decision	Admin of destination branch	sp_EmployeeTransfer_ApproveDestinationManager

11.5.4 Branches, HighAdmin governance, and reports

Method and endpoint	Access	SQL target
GET /api/branches/options	Authenticated safe catalogue	sp_Branch_ListOptions
GET /api/branches	Employee/Admin/HighAdmin	sp_Branch_GetInfo
POST /api/highadmin/managers	HighAdmin; branch ID mandatory	sp_HighAdmin_HireManager
POST /api/highadmin/managers/:id/promote	HighAdmin	sp_HighAdmin_PromoteToManager
POST /api/highadmin/managers/:id/downgrade	HighAdmin	sp_HighAdmin_DowngradeManager
POST /api/highadmin/managers/:id/suspend	HighAdmin	sp_HighAdmin_SuspendManager
POST /api/highadmin/managers/:id/fire	HighAdmin	sp_HighAdmin_FireManager
POST /api/highadmin/branches/:id/replace-manager	HighAdmin	sp_HighAdmin_ReplaceBranchManager
GET /api/reports	Authenticated report catalogue	Server-side report map
GET /api/reports/:reportKey	Role- and branch-scoped	Server-whitelisted SELECT from a protected view

Chapter 12

Frontend Architecture

12.1 Technology and file organization

The frontend intentionally uses vanilla HTML, CSS, and JavaScript. A dependency-free Node static server serves the files. Main pages are `index.html`, `customer.html`, `employee.html`, `admin.html`, and `highadmin.html`. Shared behavior is divided into:

- `config.js`: backend base URL, storage keys, role hierarchy, role-to-page routing.
- `session.js`: token/user persistence, role normalization, fixed-expiry timer, preview mode.
- `api.js`: `fetch`, Authorization header, timeout, JSON conversion, network/401 handling.
- `workspace.js`: sidebar, active section, content lifecycle, current user, responsive workspace shell.
- `features.js`: forms, tables, report viewers, actions, branch overview, account/loan/customer/employee modules.
- `ui.js`: modals, toasts, tables, date/currency formatting, CSV export, safe rendering helpers.
- role page scripts: define navigation and map section IDs to shared feature implementations.

12.2 Authentication page

The authentication page contains Sign in, Customer signup, connection settings, first HighAdmin setup, and preview mode. The left visual panel keeps only the BankManagement logo over the background image. Connection settings update the backend URL in local storage and test `/api/health`.

12.3 Frontend session behavior

The token and user object are stored in `sessionStorage` or `localStorage` depending on the persistence checkbox. The API wrapper adds `Authorization: Bearer <token>`. A local timer redirects after `ExpiresAt`, but the backend/SQL validation remains authoritative. A manipulated browser clock or disabled JavaScript cannot extend the server session.

12.4 Role-aware workspaces

Workspace	Key sections
Customer	Overview, personal accounts, transactions, personal loans/installments, profile/security.
Employee	Current branch overview, branch customers/accounts/loans read/manage scope, own accounts/transactions/loans, own employee record, transfer request, operational reports.

Admin	Current branch overview, branch customers/accounts/loans, overdue processing, branch employee actions, transfer approvals, reports, maintenance, own portfolio.
HighAdmin	Enterprise overview, manager governance, all customers/accounts/loans/employees/branches, all reports, audit/ledger, maintenance, own portfolio.

12.5 Dynamic dropdowns and branch overview

Branch and account-type choices use safe catalogue endpoints rather than free numeric text where possible. Other forms dynamically load allowed accounts, customers, employees, destination branches, and managers. Employee/Admin home pages use the `CurrentBranchID` from the validated session and display branch details and operational totals; the user cannot choose a different branch for that dashboard.

12.6 Date handling

The backend sends SQL `DATE` values as `YYYY-MM-DD` to avoid timezone day shifts, and `DATETIME` values as ISO instants. Metadata identifies date fields. The frontend uses the browser locale for readable display and the same formatter for tables, details, audit text, history, and CSV exports.

Chapter 13

End-to-End Use Cases

13.1 Customer opens an account

1. Frontend loads safe branch and account-type dropdowns.
2. Customer submits branchID, accountTypeID, and optional initial deposit.
3. Route replaces UserID with `req.user.UserID`.
4. `sp_Account_Open` resolves the linked active Customer, generates account number, validates product/branch/minimum, inserts account.
5. If deposit is supplied, it invokes the deposit procedure and returns a Pending transaction and ready time.
6. Audit records capture opening; finalization later posts the initial deposit.

13.2 Manager views and acts on branch accounts

The manager's token resolves CurrentBranchID. `GET /api/accounts` forces that branch. Lifecycle actions first assert visibility, then SQL procedures revalidate effective privilege and account state. The manager may freeze, unfreeze, change type, or close according to role/state, but cannot deposit, withdraw, transfer, finalize, reverse, or pay installments unless personally owning the relevant account/loan.

13.3 Employee hiring

A branch Admin uses `sp_Employee_Hire`; the procedure derives the manager's branch and creates Employee+EMPB only. A separate login-creation operation creates/reuses the Customer identity, creates Users, links Employee, and assigns Customer+Employee. HighAdmin manager hiring instead requires an explicit existing BranchID and atomically creates/reuses Customer, creates Employee+EMPB+Users, and assigns Customer+Employee+Admin.

13.4 Loan installment payment

The route verifies the payment account is in the caller's personal portfolio. SQL resolves installment → loan → borrower and rejects non-borrowers. It creates a Pending withdrawal transaction and stores the transaction ID in the installment. On completion, the synchronization trigger marks the installment Paid and may mark the loan Paid.

Chapter 14

Installation, Operation, and Demonstration

14.1 Database installation order

1. Create database and SQL logins.
2. Run the full table-creation script on a clean database, or apply the numbered migration installers to an existing database.
3. Install password hash function, schema-support scripts, access rules, effective-role function, triggers, procedures, reporting/security views, database users/roles, and optional Agent jobs.
4. Load realistic sample data only after all objects are installed.

14.2 Run commands

```
1 # Backend
2 npm install
3 npm run dev
4
5 # Frontend (separate terminal)
6 npm start
```

Default URLs are `http://127.0.0.1:4000` and `http://127.0.0.1:5500`. The backend `.env` contains SQL credentials, CORS origins, separate report credentials, `DB_USE_UTC=false`, and `SESSION_TTL_MINUTES=10`.

14.3 Recommended live defense demonstration

1. Show the ER diagrams and one table definition with PK/FK/CHECK constraints.
2. Login as Customer and show own accounts; attempt another customer's account and show rejection.
3. Initiate a transaction; show Pending row, ReadyToCompleteAt, available-balance hold, then finalize/process batch.
4. Login as branch manager; show current branch overview, branch-only employees/customers/accounts/loans, and account lifecycle action without transaction permission.
5. Pay an installment as borrower; show trigger-updated installment state after transaction completion.
6. Process overdue installments as manager and explain current-branch restriction.
7. Login as HighAdmin; show all reports, safe audit, branch ledger, and manager hiring with mandatory branch dropdown.
8. In code, show `request.input`, `authenticate()`, and one stored procedure with transaction/locks/audit.

Chapter 15

Testing and Verification

The Postman collection covers health, sample logins, identity resolution, effective role states, read-only fixtures, authorization boundaries, eight intentional SQL error scenarios, all fourteen reports, and optional write workflows. The latest recorded full run before the final UI-oriented patches completed 108 requests with 457 passing assertions and 9 failures. The remaining failures were concentrated in mutable optional workflows (previously transferred employee and a pending transaction before account-type change), rather than broad authentication, reports, or error-injection failures. For a final defense run, reseed/reset sample data or run stable folders separately from state-changing workflows.

15.1 Recommended additional tests

- Expired token exactly after ten minutes and explicit logout revocation.
- Parallel withdrawals/transfers to demonstrate pending holds and locks.
- Manager attempts cross-branch employee/customer/account/loan access.
- HighAdmin manager hire with missing/invalid branchID.
- Customer attempts staff report and manager attempts HighAdmin report.
- Malicious strings in search fields to demonstrate parameterization.
- Trigger tests using direct UPDATE to prove database-level protection beyond the API.

Chapter 16

Limitations and Production Hardening

Academic implementation versus production banking

The project demonstrates the required database concepts and defense-in-depth design, but it is not presented as production banking software.

- Passwords use unsalted SHA2-256 in SQL. For production, use Argon2id/bcrypt/scrypt with per-password salts in the application tier.
- Persistent bearer tokens in localStorage are vulnerable to XSS. Production should use Secure, HttpOnly, SameSite cookies, CSRF controls, device/session management, and token hashing at rest.
- Use DATETIME2 and an explicit UTC storage policy for production; the current project uses SQL Server local DATETIME plus frontend localization.
- Add encryption at rest, secrets management, TLS, structured audit details, retention policies, monitoring, backup/restore tests, and regulatory controls.
- The branch-ledger trigger records the same post-batch branch balance for multiple account deltas in one set-based statement; this is documented and could be refined with a strict sequence ledger if required.
- Centralize password policy lengths across signup, initial HighAdmin, employee login creation, and manager hiring.
- Add automated integration tests after every patch and make optional write tests idempotent with disposable employees/accounts.

Chapter 17

Questions Likely to Be Asked

How can you prove that the project does not use an ORM?

The dependency list contains `mssql` but no ORM library. `src/config/db.js` creates `sql.ConnectionPool` objects, `src/utils/procedure.js` binds explicit SQL types and calls `request.execute`, and each route names its stored procedure directly. There are no model/entity definitions or generated ORM migrations. The account, transaction, loan, employee, security, and audit rules are visible in the T-SQL source files.

How does a token reveal the role?

It does not. The opaque token locates a Sessions row; `sp_User_ValidateSession` joins the user/customer/employee/roles/current assignment and recalculates effective roles on every protected request.

Why check authorization in both backend and SQL?

Backend checks give clear HTTP behavior and prevent unnecessary calls. SQL checks protect the database even if another caller reaches a procedure or a route check is accidentally weakened. Constraints/triggers provide a third layer for invariant rules.

Why can staff have Customer role?

The project models every login as an active customer identity. Staff can therefore have personal accounts/loans, while effective Employee/Admin privileges are additional and can disappear when employment state changes.

How are customers linked to a branch?

Customer has no BranchID because a customer may bank at multiple branches. Branch correspondence is derived through `Account(CustomerID, BranchID)`, so the same customer can legitimately appear in multiple branch directories.

How is SQL injection prevented in reports with dynamic SQL?

The report key indexes a server-side constant map. View and ORDER BY strings come only from that map. Every client-supplied value remains a bound SQL parameter. The user cannot inject an identifier or SQL fragment.

What protects balances during concurrency?

Pending outgoing holds reduce available balance immediately. Procedures use transactions, `XACT_ABORT`, `TRY/CATCH`, and `UPDLOCK/HOLDLOCK` before rechecking and posting. Constraints and triggers protect final invariants.

Chapter A

Project File Map

A.1 Database

- `database/scripts/TableCreation/TableCreation_FULL_FINAL_SAFE_REBUILD.sql`: integrated destructive clean rebuild.
- `database/scripts/StoredProcedures/`: all module procedures.
- `database/scripts/Triggers/`: invariant and synchronization triggers.
- `database/scripts/Rules/`: support schema, access rules, effective-role function, overlap trigger.
- `database/security/Views/01_BankManagement_ReportingViews.sql`: reporting views.
- `database/security/DatabaseSecurityAndViews_FULL.sql`: views plus database personas/-grants.
- `database/scripts/AgentJobs/`: scheduled maintenance jobs.
- `database/install/`: SQLCMD installers and incremental patches.

A.2 Backend

- `src/app.js`, `src/server.js`: application composition and lifecycle.
- `src/config/env.js`, `src/config/db.js`: environment and SQL pools.
- `src/middleware/auth.js`: bearer validation and role middleware.
- `src/utils/procedure.js`: typed parameterized data access.
- `src/utils/access-scope.js`: own/branch/all account scope.
- `src/utils/http.js`: response/date normalization.
- `src/routes/*.routes.js`: REST modules.

A.3 Frontend

- `index.html`: authentication/signup/setup/connection page.
- `customer.html`, `employee.html`, `admin.html`, `highadmin.html`: role shells.
- `assets/js/core/`: config, session, API, UI, workspace, features, mock preview.
- `assets/js/pages/`: role navigation configuration.
- `assets/css/`: design tokens, components, auth, dashboard, responsive rules.

Chapter B

Concise Relational Schema

```
Customer(CustomerID PK, NationalID UQ, Phone UQ, Email UQ, ...)
Branch(BranchID PK, BranchCode UQ, Balance, ...)
AccountType(AccountTypeID PK, TypeName UQ, MinBalance, InterestRate, ...)
Account(AccountID PK, AccountNumber UQ, CustomerID FK, BranchID FK,
         AccountTypeID FK, Balance, OpenDate, CloseDate, AccountStatus, ...)
Employee(EmployeeID PK, NationalID UQ, JobTitle, EmpStatus, CanAccessAdmin, ...)
EMPB(EMPBID PK, EmployeeID FK, BranchID FK, StartDate, EndDate, WorkingStatus)
TransactionType(TransactionTypeID PK, TypeName UQ)
Transactions(TransactionID PK, TransactionTypeID FK, FromAccountID FK?,
              ToAccountID FK?, EmployeeID FK?, Amount, dates/status, ...)
Loan(LoanID PK, CustomerID FK, BranchID FK, LoanAmount, InterestRate, dates/status)
Installment(InstallmentID PK, LoanID FK, PaymentTransactionID FK?, DueDate, ...)
BranchLedger(BranchLedgerID PK, BranchID FK, TransactionID FK?, delta/balance/date)
Roles(RoleID PK, RoleName UQ)
Users(UserID PK, Username UQ, PasswordHash, CustomerID FK UQ?, EmployeeID FK UQ?)
UserRoles(UserRoleID PK, UserID FK, RoleID FK, UQ(UserID,RoleID))
Sessions(SessionID PK, UserID FK, SessionToken UQ, LoginTime, ExpiresAt, ...)
AuditLog(AuditID PK, UserID FK?, ActionType, TableName, RecordID, ActionDate, Details)
EmployeeTransferRequest(TransferRequestID PK, EmployeeID FK, FromBranchID FK,
                        ToBranchID FK, requester/approvers FK, decisions/status/dates)
```

Chapter C

Selected Code Excerpts

C.1 Server-defined report whitelist

```
1 const REPORT_VIEWS = {
2   'audit-trail': {
3     view: 'vw_AuditTrail_Safe',
4     poolName: 'audit',
5     roles: ['HighAdmin'],
6     orderBy: 'R.ActionDate DESC, R.AuditID DESC',
7     dateFields: { ActionDate: 'datetime' }
8   }
9 };
```

C.2 Trusted identity overrides client identity

```
1 values: {
2   ...req.body,
3   EmployeeID: req.user.EmployeeID || null,
4   UserID: req.user.UserID
5 }
```

The trusted values occur after `...req.body`; therefore a malicious body cannot override `UserID/EmployeeID`.

C.3 Fixed session expiration

```
1 SET @LoginTime = GETDATE();
2 SET @ExpiresAt = DATEADD(MINUTE, @SessionTtlMinutes, @LoginTime);
3 SET @SessionToken = CONVERT(NVARCHAR(36), NEWID())
4   + N'-' + CONVERT(NVARCHAR(36), NEWID());
5 INSERT INTO dbo.Sessions(UserID, SessionToken, LoginTime, ExpiresAt, IsActive)
6 VALUES(@UserID, @SessionToken, @LoginTime, @ExpiresAt, 1);
```

Presentation Summary

BankManagement is designed as a SQL-centered application. The database is not a passive storage layer: it owns identity state, effective roles, temporal branch relationships, financial invariants, transaction atomicity, pending holds, loan/installment synchronization, audit, reporting, and scheduled maintenance. The backend is a thin but important secure API layer that validates sessions, applies broad role/scope rules, binds parameters, and exposes structured endpoints. The frontend is a role-aware client that presents only relevant workflows, but it is never treated as a security boundary.

The following points summarize the main design decisions:

1. The schema is relational and normalized, with controlled audit/aggregate denormalization.
2. No ORM replaces SQL; all critical operations are explicit T-SQL procedures, views, triggers, functions, and jobs.
3. SQL injection is prevented by typed parameter binding and server-owned identifier whitelists.
4. Authorization is dynamic: assigned roles become effective only when current customer/employee/manager state permits them.
5. Ownership and branch scope are derived from the validated session, not trusted client IDs.
6. Financial operations are owner-only and use pending holds, transaction locks, finalization, triggers, and audit.
7. Sessions are server-side, revocable, and expire after a fixed ten minutes.
8. Reporting uses least-privilege SQL pools and safe views, with newest-first ordering and correct date semantics.